

Knowledge Based Engineering (KBE)

AE4204

Dr.ir. G. La Rocca

Flight Performance & Propulsion

Group (FPP)

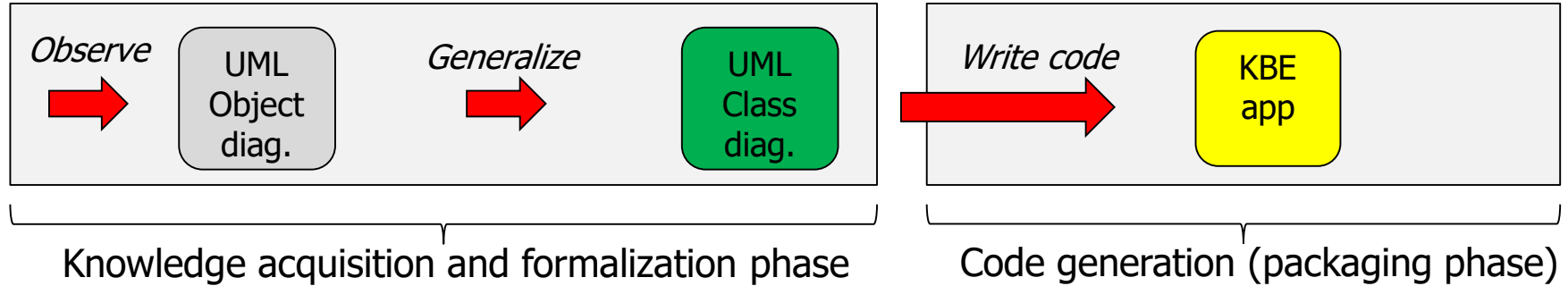
Lecture 7

- *UML extension for KBE*
- *UML activity diagrams*

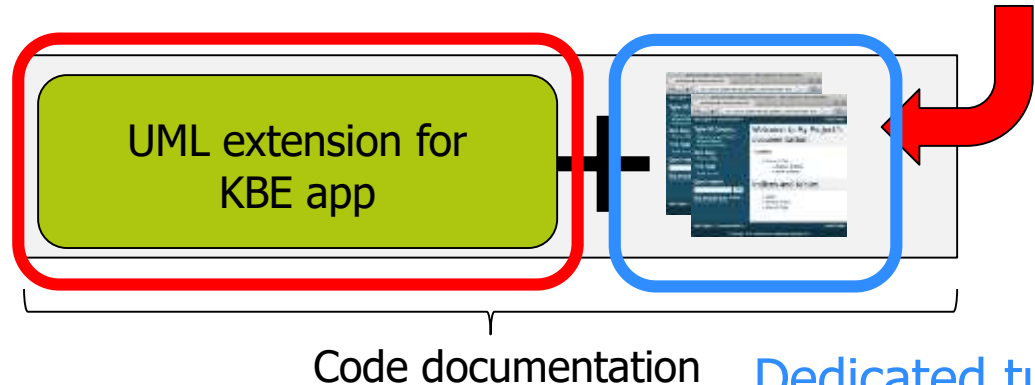


UML extension for KBE

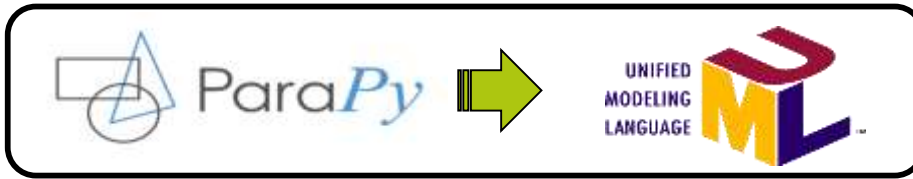
UML modeling and KBE



This lecture



UML modeling for KBE

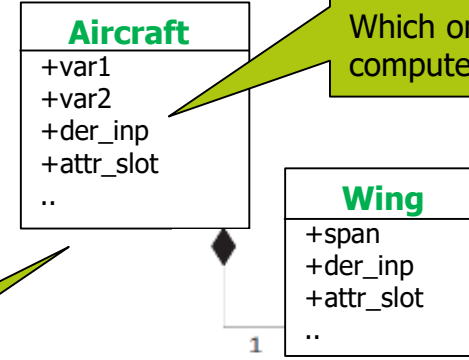


```
class Aircraft(Base):  
    #: required input slot  
    var1=Input()  
    #: optional input slot  
    var2=Input(value)  
    @Input  
    def der_inp (self):  
        return  
    @Attribute  
    def attr_slot(self):  
        return  
    @Part  
    def main_wing(self):  
        return Wing(span=10)
```

```
class Wing(Base):  
    #: required input slot  
    span=Input(15)  
    @Input  
    def der_inp (self):  
        return  
    @Attribute  
    def attr_slot (self):  
        return  
    @Part  
    def aileron(self):  
        return Movable()
```

What are the names of the object?

UML Class diagram



Which slots are settable?

Which ones computed?

UML Object diagram



UML extension for KBE



ParaPy



Extended(

UNIFIED
MODELING
LANGUAGE



)

```
class Aircraft(Base):
```

```
#: required input slot  
var1=Input()  
#: optional input slot  
var2=Input(value)
```

```
@Input  
def der_inp(self):  
    return
```

```
@Attribute  
def attr_slot(self):  
    return
```

```
@Part  
def main_wing(self):  
    return wing(span=10)
```

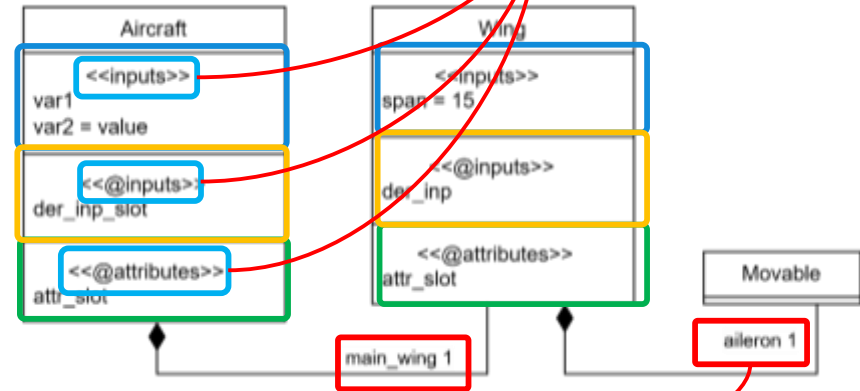
```
class Wing(Base):
```

```
# required input slot  
span=Input(15)
```

```
@Input  
def der_inp(self):  
    return
```

```
@Attribute  
def attr_slot(self):  
    return
```

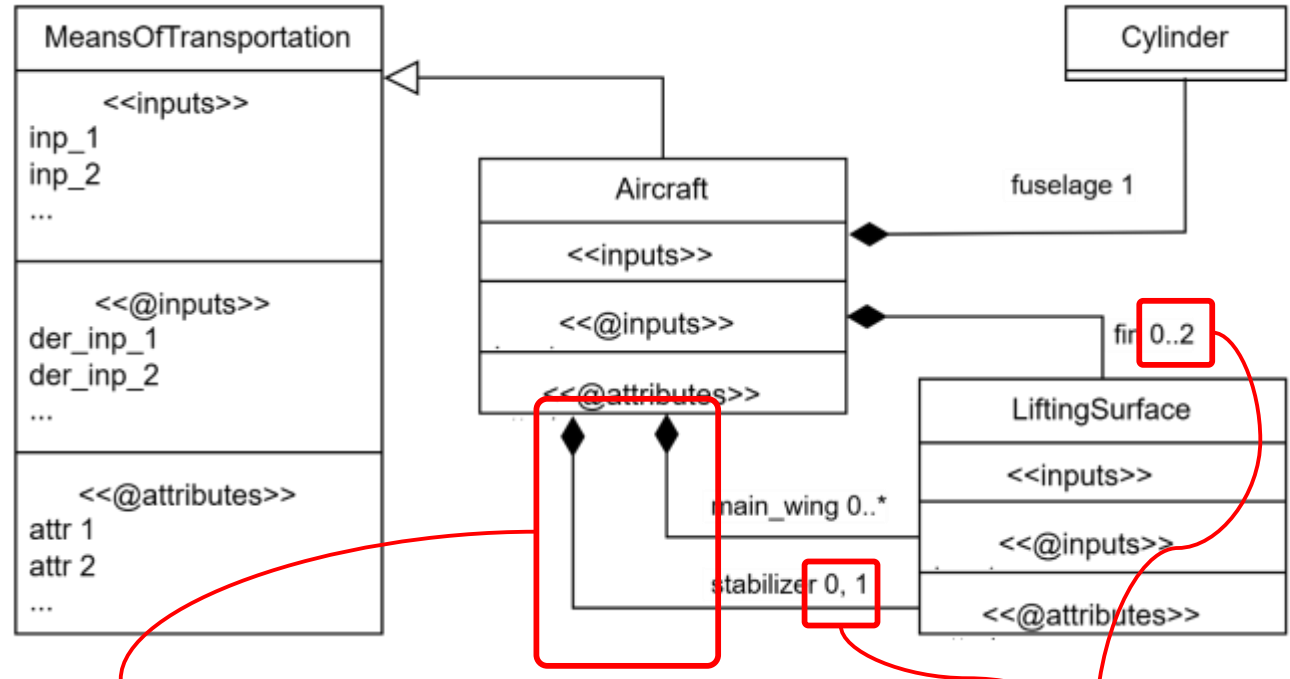
```
@Part  
def aileron(self):  
    return Movable()
```



Part name

`<<stereotype>>`

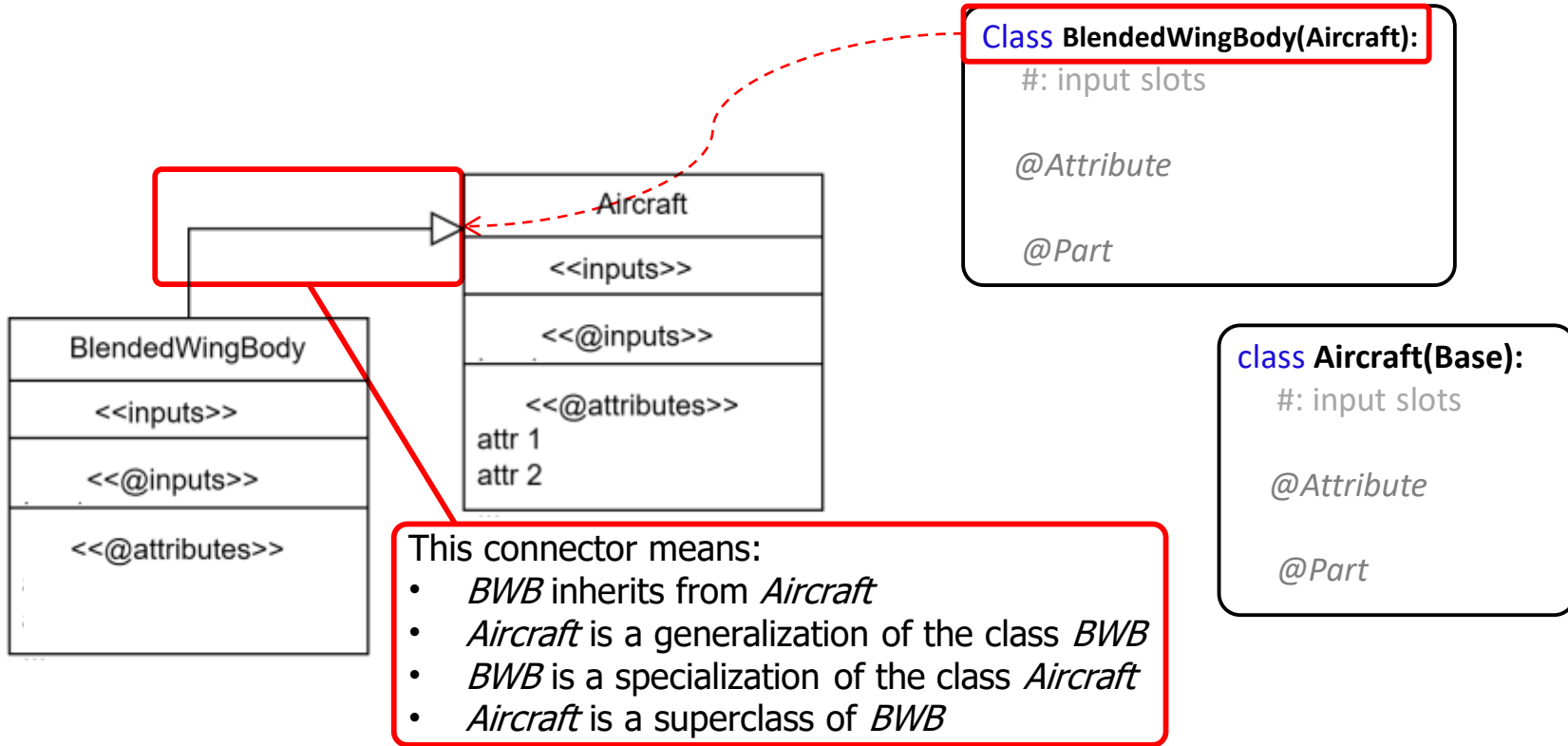
UML extension for KBE



Multiple composition connectors to show the same component class is used to create different Aircraft parts

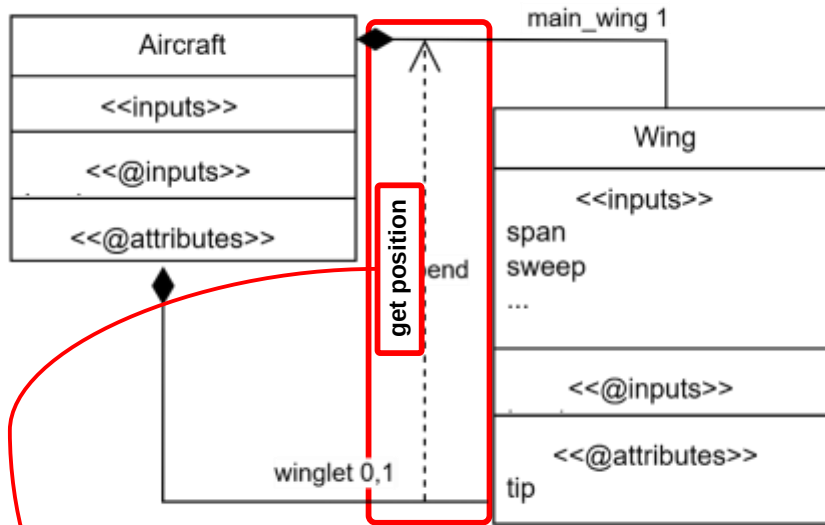
Multiplicity label → quantify

UML extension for KBE



UML extension for KBE

Both child-objects *main_wing* and *winglet* are instances of the same class *Wing*



The part *winglet* depends on the part *main_wing* (because of its positioning)

```
class Aircraft():
```

```
    #: input slots
```

```
    @Attribute
```

```
    @Part
```

```
    def main_wing(self):
```

```
        return Wing(span=self.span,  
                    sweep=self.sweep)
```

```
    @Part
```

```
    def winglet(self):
```

```
        return Wing(position=  
                    self.position(self.main_wing.tip))
```

```
class Wing():
```

```
    #: input slots
```

```
    @Attribute
```

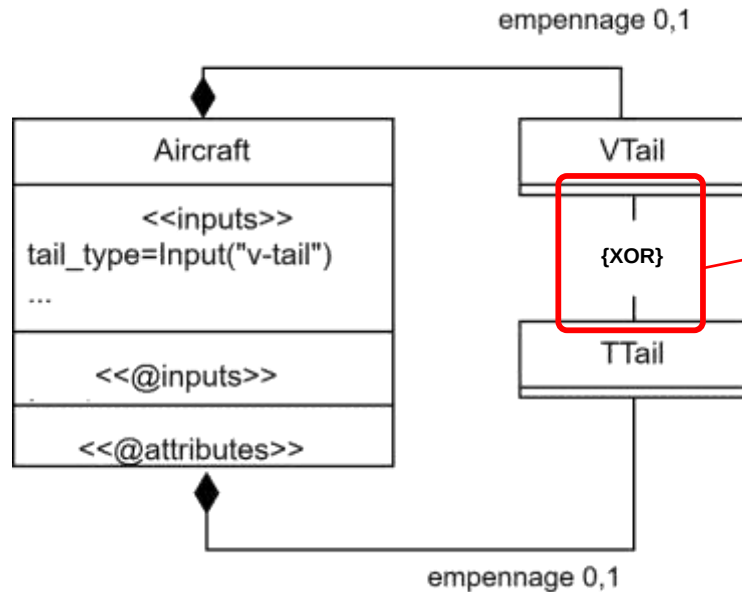
```
    def tip(self):
```

```
        return
```

```
    @Part...
```

UML extension for KBE

The class to instantiate for generating the part *empennage* is established **dynamically**, by means of an IF-THEN rule



```
class Aircraft():
```

```
    #: input slots
```

```
    tail_type=Input("v-tail" )
```

```
    @Attribute
```

```
    @Part
```

```
    def empennage(self):
```

```
        return DynamicType(
```

```
            type=VTail if self.tail_type=="v-tail"
```

```
            else TTail,
```

```
            ...)
```

```
class TTail():
```

```
    #: input slots
```

```
    @Attribute
```

```
    @Part
```

```
class VTail():
```

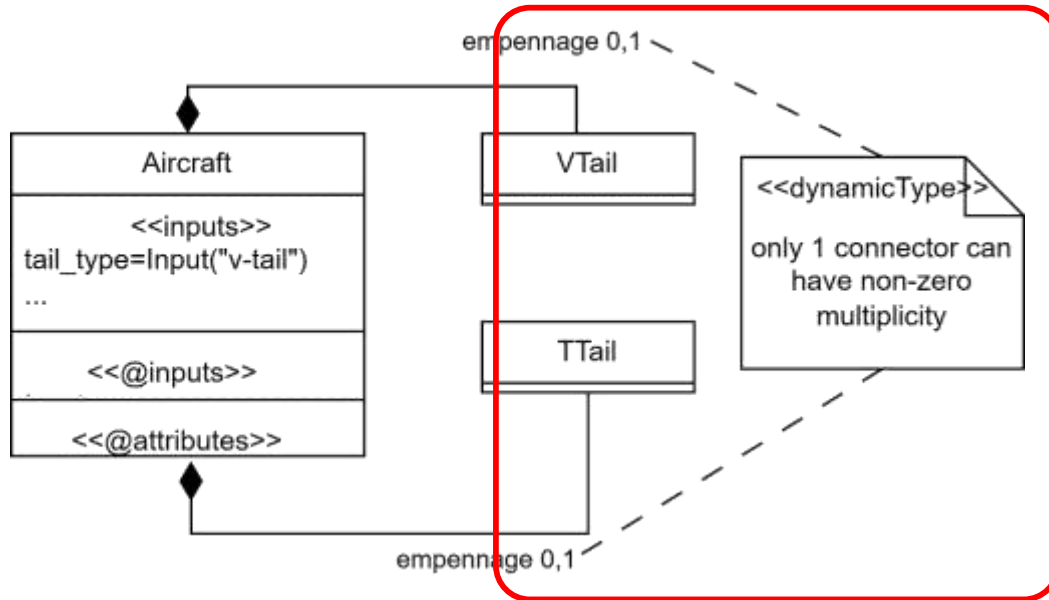
```
    #: input slots
```

```
    @Attribute
```

```
    @Part
```

UML extension for KBE

The class to instantiate for generating the part *empennage* is established **dynamically**, by means of an IF-THEN rule



```
class Aircraft():
```

```
    #: input slots
```

```
    tail_type=Input("v-tail" )
```

```
    @Attribute
```

```
    @Part
```

```
    def empennage(self):
```

```
        return DynamicType(
```

```
            type=VTail if self.tail_type=="v-tail"
```

```
            else TTail,
```

```
            ...)
```

```
class TTail():
```

```
    #: input slots
```

```
    @Attribute
```

```
    @Part
```

```
class VTail():
```

```
    #: input slots
```

```
    @Attribute
```

```
    @Part
```

UML extension for KBE - recap

- **Extends standard UML class diagrams** with stereotypes tailored to KBE (e.g., <<inputs>>, <<@inputs>>, <<@attributes>>) to reflect ParaPy class elements
- **Shows both class structure and generated object structure in one diagram** by naming instances on composition connectors (e.g., fuselage : Cylinder)
- **Captures dependency relations between components** using labeled dependency connectors (e.g., get position, get attribute X), while omitting implied parent→child dependencies to reduce clutter
- **Represents runtime type selection** (DynamicType) by modeling alternative classes and constraining them so that only one class is instantiated (e.g., via {or} node or multiplicity constraint)
- **Maintains UML compatibility** by using stereotypes, constraints, multiplicities, and associations, effectively functioning as a custom UML profile for KBE.

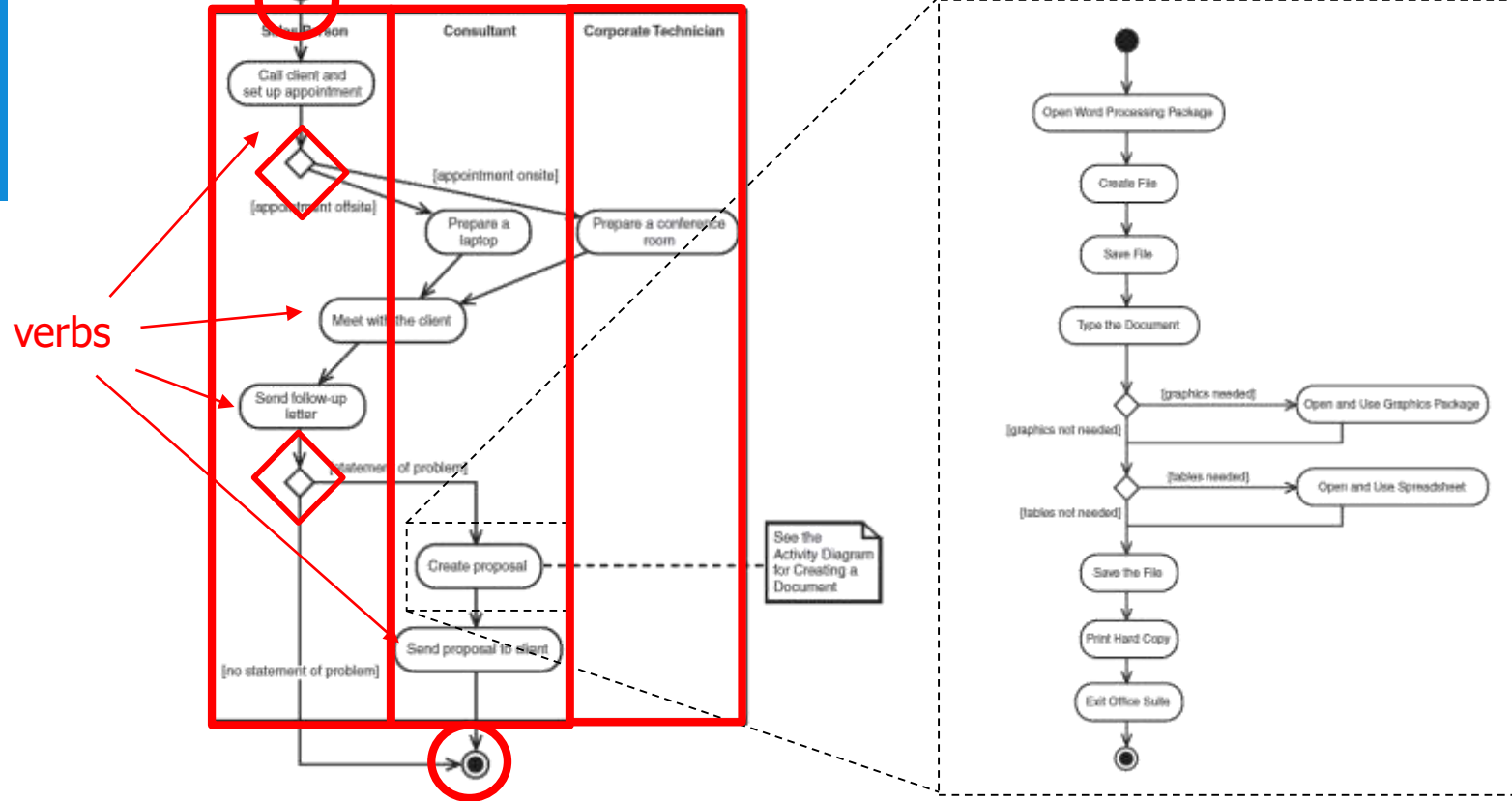
UML extension for KBE – why useful

- **Bridges the gap between UML and KBE frameworks (e.g., ParaPy)** by explicitly modeling object trees, dependencies, and configuration logic in a single diagram
- **Improves readability of complex generative models**, avoiding multiple separate UML diagrams (class, object, dependency)
- **Highlights essential knowledge dependencies and configuration choices**, making model behavior easier to understand, debug, and communicate
- **Supports documentation of KBE systems**, helping engineers grasp how inputs, rules, and parts interact to generate engineering models

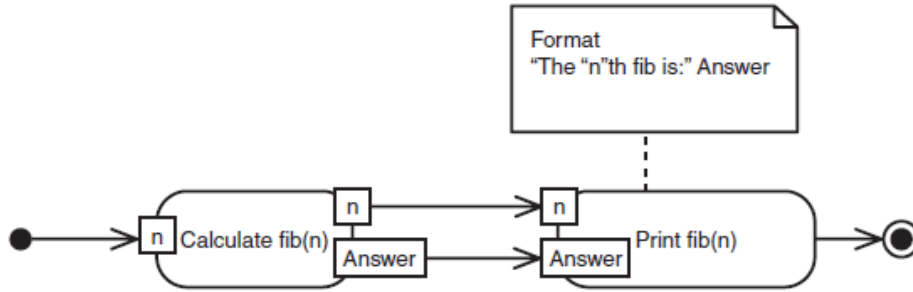
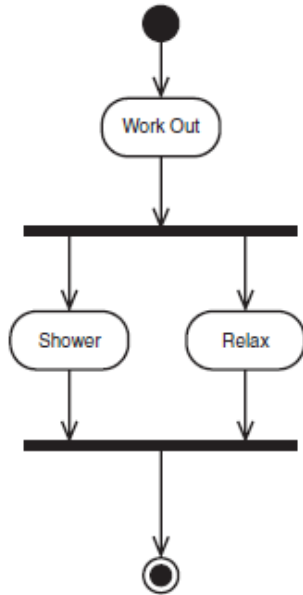
UML activity diagrams

- **To explain engineering logic**
- **To help document design reasoning**
- **To structure the knowledge capture process**

UML activity diagrams



UML activity diagrams



- Use activity diagrams to model **conceptual process knowledge (i.e. engineering design logic), but...**
- be aware they are **not reflecting the demand-driven execution mechanism of a KBE system**

Main reference material for this lecture

1. **Book** *MDO supported by KBE* by Wiley: **Chapter 9 on KBE**
2. **Journal paper** *Knowledge based engineering: Between AI and CAD. Review of a language based technology to support engineering design*, G. La Rocca,, *Journal of Advanced Engineering Informatics*, Vol. 26, pp.159–179, 2012.
3. **On UML** (*Object Oriented modeling Class and Object diagrams; A summary of UML notation*)
4. **ParaPy documentation**
5. **Scratchpad for drawio**



Ref 2-3 available in the section Reading Material